TRAINING AND SCALING LANGUAGE MODELS: FROM FIRST PRINCIPLES TO EFFICIENT SERVING

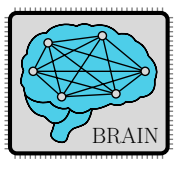
Welcome to Training & Scaling Language Models

Course Presentation: What We Will Do, Organization, and Projects

BRAIN team

IMT Atlantique · January 2027

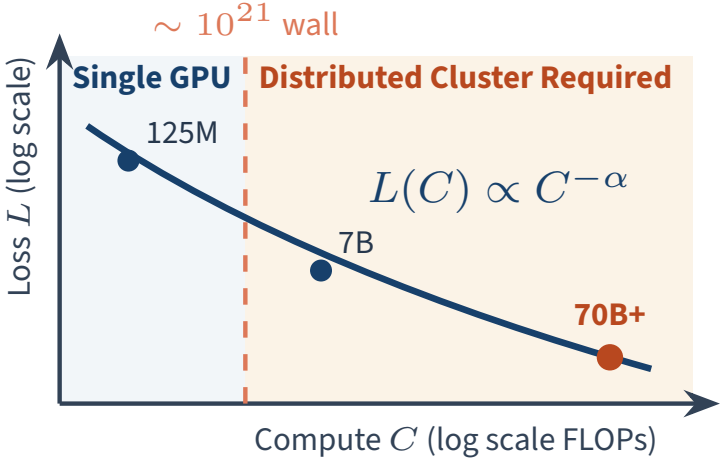




Context: Why distribute? The empirical mandate of scale

Scaling laws (Compute laws)

- **Empirical power law**: Test loss drops predictably across > 8 orders of magnitude:
  $\mathbf{L(C) \approx (C_c/C)^{\alpha_C}}, \quad C \approx 6ND$
- **Predictable returns**: More FLOPs buys lower perplexity, reasoning, and benchmark accuracy.
- **Scaled performance**: Scaling compute unlocks capabilities smaller models cannot reach.



Frontier scale ($\sim 10^{25}$ FLOPs):

$1 \times$ H100: > 800 **years**

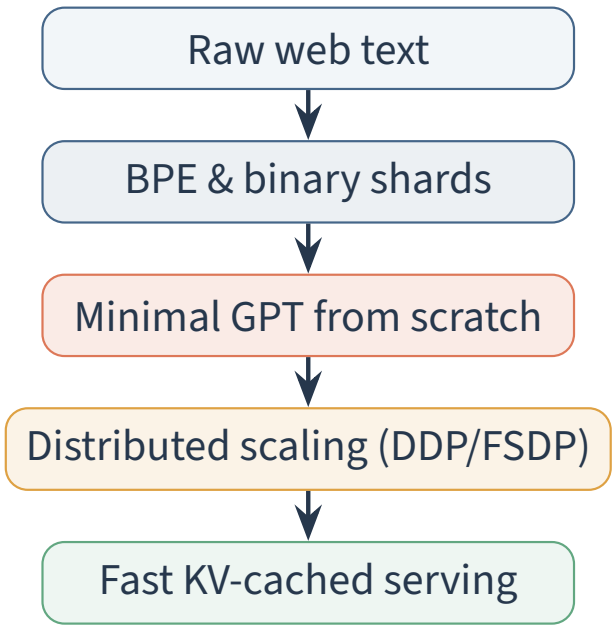
$\downarrow$

$16\text{k} \times$ H100: ≈ 18 **days**

What will you build in this course?

Not an API-calling tutorial.
We open the engine from first principles:

- Start with raw, noisy web text.
- Build a decoder-only Transformer from scratch.
- Train, profile, and push a single GPU to its limits.
- Scale across multiple GPUs and nodes.
- Serve the trained model with low latency.



The governing question

Language models are limited by hardware physics, not just algorithms:

1. Compute wall

Tensor Cores require high arithmetic intensity:

$$\frac{\text{FLOPs}}{\text{Bytes transferred}}$$

Low intensity starves execution units.

2. Memory wall

Weights, optimizer states, activations, and KV cache compete for device RAM.

3. Network wall

NVLink and InfiniBand bandwidth limits distributed collective speed.

The governing question throughout

How do we turn raw text and finite compute into a language model that trains, scales, and serves efficiently?

The 3 phases of the course

40 contact hours organized into 32 sessions of 1h15:

Phase 1: Build (15h)

Single-GPU foundation

- Transformer architecture
- Training loop & autograd
- Data pipeline & BPE
- Profiling, MFU, FlashAttn

Phase 2: Scale (15h)

Distributed systems

- Mid-course paper talks
- DDP (all-reduce)
- FSDP & ZeRO sharding
- Tensor & 5D parallelism

Phase 3: Serve (10h)

Inference & synthesis

- KV-cache decode
- Continuous batching
- Frontier architectures
- Final project defense

Each topic connects theory directly to measurable code.

How sessions work: theory + hands-on practice

Theory & derivations

We unpack the mechanics: tensor shapes, autograd graphs, collective communication costs, and roofline models.

Guided implementation

You immediately build, test, and profile the mechanism using our companion PyTorch repository.

No waiting on progress bars

- Class time is for **building, diagnosing, and measuring**.
- Long-running training jobs run asynchronously on the GPU cluster between sessions.
- Next class begins by analyzing the checkpoints, curves, and profiler traces.

The two projects

1. The Continuous Laboratory

The shared codebase everyone builds

- In-class hands-on backbone.
- Every team builds the same evolving system from module 1 to 11.
- Validated by unit tests, shape checks, and profiler reports.
- Your sandbox and toolkit.

2. The Reproduction Project

The graded team challenge (65%)

- Pick a recent top-tier research paper (systems, scaling, data, efficiency).
- Scale down and verify **one precise claim** with controlled evidence.
- Supported by 4 dedicated in-class studio hackathons with instructors.

Reproduction project: what we expect

What NOT to do

Do not try to retrain a 70B model or reproduce an entire 20-page paper.

Valid reproduction approaches

- Test a memory or throughput claim under synthetic workloads.
- Test an architectural tweak on a smaller model (e.g. 124M).
- Verify a scaling law or ablation trend under a fixed compute budget.

Negative results welcome

A rigorous experiment that explains *why* a published claim did not hold under controlled settings gets full credit.

Scientific rigor

Controlled variables, verified baselines, and clear ablations matter far more than raw model size.

Grading and assessment

We grade engineering rigor, correct implementations, and controlled reasoning:

Component	Weight	Format
Pre-session quizzes	**15%**	5-min online checks before each teaching day; verifies reading & core concepts
Mid-course paper talk	**20%**	8-min presentation + 4-min Q&A; explain the paper's mechanism & proposed test
Reproduction project	**65%**	Reproducible code, short technical report ($\leq$ 5 pages), and team defense with individual Q&A

Quizzes drop the lowest scores to accommodate occasional technical issues or absences.

Course toolkit and logistics

- **Companion repo**: `llm-course-companion`
- Clean, modular PyTorch implementation for each module.
- Automated test suites:
  - Tensor shape invariants
  - Causal masking checks
  - Tiny-batch overfit ($L < 0.1$)
  - Checkpoint resume determinism
- Fast dependency management with `uv`.
- Dedicated GPU compute nodes for training runs.

Sanity check before Session 1

```
# Clone & install
git clone <repo-url>
cd llm-course-companion
uv sync

# Check GPU & tests
nvidia-smi
uv run pytest tests/
```

Summary: our contract

1. **First principles**: No black boxes. We understand every tensor shape, CUDA kernel launch, and collective communication.
2. **Measure before optimizing**: Trust profiler traces and MFU, not intuition.
3. **Build together, explore in teams**: Master the fundamentals in the continuous lab, then test cutting-edge research in your reproduction project.
4. **Rigorous science**: Real engineering is about reproducible baselines and honest analysis of trade-offs.

Let’s build a language model!